

```
C:\Windows\system32\cmd.e: X + v
(base) C:\Users\bharg>python -c "import tensorflow as tf; print(tf.__version__)"
WARNING: All log messages before absl::InitializeLog() is called are written to STDERR
I0000 00:00:1784179292.608638    6552 port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to floating-point round-off errors from different computation orders. To turn them off, set the environment variable 'TF_ENABLE_ONEDNN_OPTS=0'.
WARNING: All log messages before absl::InitializeLog() is called are written to STDERR
I0000 00:00:1784179295.917953    6552 port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to floating-point round-off errors from different computation orders. To turn them off, set the environment variable 'TF_ENABLE_ONEDNN_OPTS=0'.
2.21.0

(base) C:\Users\bharg>python --version
Python 3.13.9

(base) C:\Users\bharg>python -c "import numpy; print(numpy.__version__)"
2.3.5

(base) C:\Users\bharg>python -c "import matplotlib; print(matplotlib.__version__)"
3.10.6

(base) C:\Users\bharg>python -c "import keras; print(keras.__version__)"
WARNING: All log messages before absl::InitializeLog() is called are written to STDERR
I0000 00:00:1784179376.560602   16756 port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to floating-point round-off errors from different computation orders. To turn them off, set the environment variable 'TF_ENABLE_ONEDNN_OPTS=0'.
WARNING: All log messages before absl::InitializeLog() is called are written to STDERR
I0000 00:00:1784179380.553186   16756 port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to floating-point round-off errors from different computation orders. To turn them off, set the environment variable 'TF_ENABLE_ONEDNN_OPTS=0'.
3.15.0

(base) C:\Users\bharg>python -c "import gymnasium; print(gymnasium.__version__)"
1.3.0

(base) C:\Users\bharg>
```

C:\Users\bharg\OneDrive\Desktop\exp2.py

exp2.py x

```
1 import gymnasium as gym
2
3 environments = ["FrozenLake-v1", "CartPole-v1", "MountainCar-v0"]
4
5 for env_name in environments:
6
7     print("\n=====")
8     print("Environment:", env_name)
9     print("=====")
10
11     env = gym.make(env_name)
12
13     state, info = env.reset()
14
15     print("Initial State:", state)
16
17     for i in range(5):
18
19         action = env.action_space.sample()
20
21         next_state, reward, terminated, truncated, info = env.step(action)
22
23         print("\nStep:", i + 1)
24         print("Action:", action)
25         print("Next State:", next_state)
26         print("Reward:", reward)
27         print("Episode Ended:", terminated or truncated)
28
29         if terminated or truncated:
30             break
31
32     env.close()
```

C:\Users\bharg\OneDrive\Desktop



Code not profiled yet

Profile your code to explore which functions and methods took the longest to run and were called the most, and find out where to optimize it.

Help Variable Explorer Debugger Profiler Plots Files

Console 1/A x

```
Episode Ended: False
Step: 3
Action: 0
Next State: [-0.5826728 -0.00173785]
Reward: -1.0
Episode Ended: False

Step: 4
Action: 2
Next State: [-5.8296990e-01 -2.9710916e-04]
Reward: -1.0
Episode Ended: False

Step: 5
Action: 1
Next State: [-5.8282405e-01 1.4582345e-04]
Reward: -1.0
Episode Ended: False
```

In [2]:

IPython Console History

InLine Conda: anaconda3 (Python 3.13.9) LSP: Python Line 32, Col 16 UTF-8 CRLF RW Mem 84%



question 3.py - C:/Users/bharg/OneDrive/Desktop/assignment/question 3.py (3.11.6)

File Edit Format Run Options Window Help

```
import random

states = ["A", "B", "C", "Goal"]
actions = ["Left", "Right"]

state = "A"

print("Initial State:", state)

while state != "Goal":

    action = random.choice(actions)

    print("\nCurrent State:", state)
    print("Action:", action)

    if state == "A":
        state = "B"

    elif state == "B":
        state = "C"

    elif state == "C":
        state = "Goal"

    reward = 10 if state == "Goal" else -1

    print("Next State:", state)
    print("Reward:", reward)

print("\nGoal Reached!")
print("Episode Finished")
```

Ln: 32 Col: 25

IDLE Shell 3.11.6

File Edit Shell Debug Options Window Help

Python 3.11.6 (tags/v3.11.6:8b6ee5b, Oct 2 2023, 14:57:12) [MSC v.1935 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.

>>>

===== RESTART: C:/Users/bharg/OneDrive/Desktop/assignment/question 3.py =====

Initial State: A

Current State: A

Action: Right

Next State: B

Reward: -1

Current State: B

Action: Right

Next State: C

Reward: -1

Current State: C

Action: Left

Next State: Goal

Reward: 10

Goal Reached!

Episode Finished

>>>

Ln: 24 Col: 0

```
import random
```

```
grid = [
    ['S', '.', 'D', '.', '.'],
    ['.', 'X', '.', 'D', '.'],
    ['.', '.', '.', 'X', '.'],
    ['D', '.', '.', '.', '.'],
    ['.', 'X', '.', 'D', '.']
]
```

```
row = 0
col = 0
reward = 0
```

```
for i in range(20):
```

```
    move = random.choice(["UP", "DOWN", "LEFT", "RIGHT"])
```

```
    if move == "UP" and row > 0:
        row -= 1
    elif move == "DOWN" and row < 4:
        row += 1
    elif move == "LEFT" and col > 0:
        col -= 1
    elif move == "RIGHT" and col < 4:
        col += 1
```

```
    print("Move:", move)
    print("Position:", (row, col))
```

```
    if grid[row][col] == "D":
        reward += 1
        print("Cleaned Dirt (+1)")
        grid[row][col] = '.'
```

```
    elif grid[row][col] == "X":
        reward -= 1
        print("Hit Obstacle (-1)")
```

```
    print("Total Reward:", reward)
```

Ln: 1 Col: 0

```
Position: (1, 1)
Hit Obstacle (-1)
Total Reward: -4
```

```
Move: LEFT
Position: (1, 0)
Total Reward: -4
```

```
Move: RIGHT
Position: (1, 1)
Hit Obstacle (-1)
Total Reward: -5
```

```
Move: UP
Position: (0, 1)
Total Reward: -5
```

```
Move: RIGHT
Position: (0, 2)
Cleaned Dirt (+1)
Total Reward: -4
```

```
Move: DOWN
Position: (1, 2)
Total Reward: -4
```

```
Move: DOWN
Position: (2, 2)
Total Reward: -4
```

```
Move: DOWN
Position: (3, 2)
Total Reward: -4
```

```
Move: LEFT
Position: (3, 1)
Total Reward: -4
```

```
Final Reward = -4
```

>>>

Ln: 92 Col: 0

question 5.py - C:/Users/bharg/OneDrive/Desktop/assignment/question 5.py (3.11.6)

File Edit Format Run Options Window Help

```
gamma = 0.9
rewards = [5, 10, 15]
values = [0, 0, 0]

print("Bellman Expectation Equation")

for i in range(10):
    for j in range(3):
        values[j] = rewards[j] + gamma * values[j]

    print("Iteration", i + 1, ":", values)
```

Ln: 16 Col: 0

IDLE Shell 3.11.6

File Edit Shell Debug Options Window Help

Python 3.11.6 (tags/v3.11.6:8b6ee5b, Oct 2 2023, 14:57:12) [MSC v.1935 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.

>>>

===== RESTART: C:/Users/bharg/OneDrive/Desktop/assignment/question 5.py =====

Bellman Expectation Equation

Iteration 1 : [5.0, 10.0, 15.0]

Iteration 2 : [9.5, 19.0, 28.5]

Iteration 3 : [13.55, 27.1, 40.650000000000006]

Iteration 4 : [17.195, 34.39, 51.58500000000001]

Iteration 5 : [20.4755, 40.951, 61.42650000000001]

Iteration 6 : [23.42795, 46.8559, 70.28385]

Iteration 7 : [26.085155, 52.17031, 78.255465]

Iteration 8 : [28.4766395, 56.953279, 85.4299185]

Iteration 9 : [30.62897555, 61.2579511, 91.88692665]

Iteration 10 : [32.566077995, 65.13215599, 97.698233985]

>>>

Ln: 16 Col: 0


```
question 6.py - C:/Users/bharg/OneDrive/Desktop/assignment/question 6.py (3.11.6)
File Edit Format Run Options Window Help
import random

actions = ["Left", "Right", "Up", "Down"]
values = [5, 10, 3, 8]

print("Random Strategy")
for i in range(5):
    action = random.choice(actions)
    print("Action:", action)

print("\nGreedy Strategy")
best_action = actions[values.index(max(values))]
for i in range(5):
    print("Action:", best_action)

print("\nEpsilon-Greedy Strategy")
epsilon = 0.2

for i in range(5):
    if random.random() < epsilon:
        action = random.choice(actions)
    else:
        action = best_action

    print("Action:", action)
```

Ln: 27 Col: 0

```
IDLE Shell 3.11.6
File Edit Shell Debug Options Window Help
Python 3.11.6 (tags/v3.11.6:8b6ee5b, Oct 2 2023, 14:57:12) [MSC v.1935 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: C:/Users/bharg/OneDrive/Desktop/assignment/question 6.py =====
Random Strategy
Action: Left
Action: Left
Action: Up
Action: Right
Action: Left

Greedy Strategy
Action: Right
Action: Right
Action: Right
Action: Right
Action: Right

Epsilon-Greedy Strategy
Action: Right
Action: Right
Action: Right
Action: Right
Action: Right
>>>
```

Ln: 25 Col: 0

```
question 7.py - C:/Users/bharg/OneDrive/Desktop/assignment/question 7.py (3.11.6)
File Edit Format Run Options Window Help
total_reward = 0

for i in range(10):
    if random.random() < epsilon:
        action = random.randint(0, 2)
    else:
        action = rewards.index(max(rewards))

    reward = rewards[action]
    total_reward += reward

    print("Step:", i+1, "Action:", action, "Reward:", reward)

print("Total Reward:", total_reward)

print("\nUpper Confidence Bound (UCB)")

counts = [1, 1, 1]
values = rewards.copy()
total_reward = 0

for i in range(10):

    ucb = []

    for j in range(3):
        value = values[j] + math.sqrt((2 * math.log(i + 4)) / counts[j])
        ucb.append(value)

    action = ucb.index(max(ucb))

    reward = rewards[action]
    total_reward += reward
    counts[action] += 1

    print("Step:", i+1, "Action:", action, "Reward:", reward)

print("Total Reward:", total_reward)
```

Ln: 49 Col: 0

```
IDLE Shell 3.11.6
File Edit Shell Debug Options Window Help

Python 3.11.6 (tags/v3.11.6:8b6ee5b, Oct 2 2023, 14:57:12) [MSC v.1935 64 bit (
AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.

>>>
===== RESTART: C:/Users/bharg/OneDrive/Desktop/assignment/question 7.py =====
Epsilon-Greedy
Step: 1 Action: 1 Reward: 10
Step: 2 Action: 1 Reward: 10
Step: 3 Action: 0 Reward: 5
Step: 4 Action: 1 Reward: 10
Step: 5 Action: 1 Reward: 10
Step: 6 Action: 1 Reward: 10
Step: 7 Action: 1 Reward: 10
Step: 8 Action: 1 Reward: 10
Step: 9 Action: 1 Reward: 10
Step: 10 Action: 1 Reward: 10
Total Reward: 95

Upper Confidence Bound (UCB)
Step: 1 Action: 1 Reward: 10
Step: 2 Action: 1 Reward: 10
Step: 3 Action: 1 Reward: 10
Step: 4 Action: 1 Reward: 10
Step: 5 Action: 1 Reward: 10
Step: 6 Action: 1 Reward: 10
Step: 7 Action: 1 Reward: 10
Step: 8 Action: 1 Reward: 10
Step: 9 Action: 1 Reward: 10
Step: 10 Action: 1 Reward: 10
Total Reward: 100

>>>
```

Ln: 30 Col: 0

qestion 8.py - C:/Users/bharg/OneDrive/Desktop/assignment/qestion 8.py (3.11.6)

File Edit Format Run Options Window Help

```
import random
import matplotlib.pyplot as plt

episodes = 10

episode_rewards = []
cumulative_rewards = []

total_reward = 0

for episode in range(episodes):

    reward = random.randint(1, 10)

    episode_rewards.append(reward)

    total_reward += reward

    cumulative_rewards.append(total_reward)

print("Episode Rewards:", episode_rewards)
print("Cumulative Rewards:", cumulative_rewards)

plt.figure(figsize=(8,5))

plt.plot(range(1, episodes + 1), episode_rewards,
         marker='o', label="Episode Reward")

plt.plot(range(1, episodes + 1), cumulative_rewards,
         marker='s', label="Cumulative Reward")

plt.xlabel("Episode")
plt.ylabel("Reward")
plt.title("Reward Visualization in Reinforcement Learning")
plt.legend()

plt.grid(True)

plt.show()
```

Ln: 40 Col: 0

IDLE Shell 3.11.6

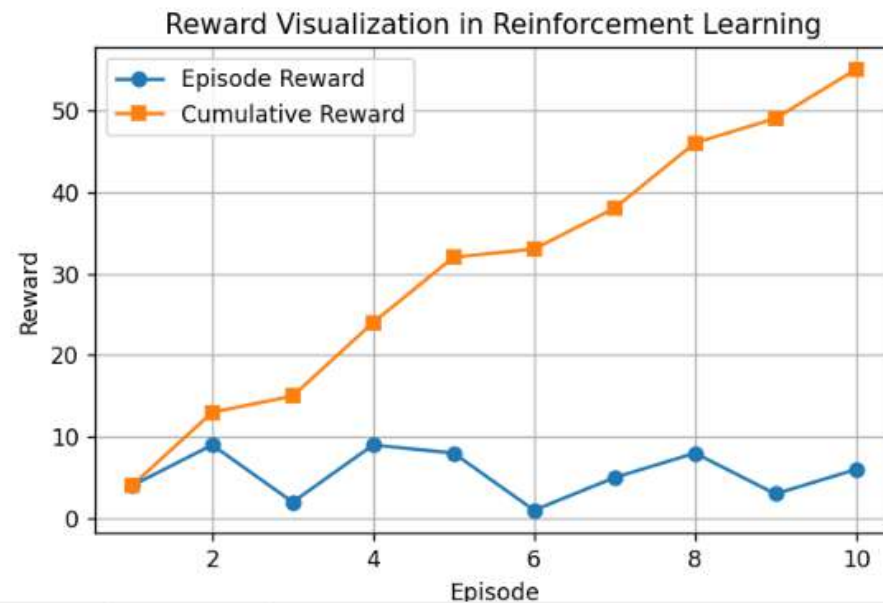
File Edit Shell Debug Options Window Help

Python 3.11.6 (tags/v3.11.6:8b6ee5b, Oct 2 2023, 14:57:12) [MSC v.1935 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.

>>>

```
===== RESTART: C:/Users/bharg/OneDrive/Desktop/assignment/qestion 8.py =====
Episode Rewards: [4, 9, 2, 9, 8, 1, 5, 8, 3, 6]
Cumulative Rewards: [4, 13, 15, 24, 32, 33, 38, 46, 49, 55]
```

Figure 1



Ln: 5 Col: 0


```
[3]: import tensorflow as tf
from tensorflow import keras

model = keras.Sequential([
    keras.layers.Dense(16, activation="relu", input_shape=(4,)),
    keras.layers.Dense(16, activation="relu"),
    keras.layers.Dense(1)
])

model.compile(optimizer="adam",
              loss="mse",
              metrics=["accuracy"])

model.summary()
```

Model: "sequential_2"

Layer (type)	Output Shape	Param #
dense_6 (Dense)	(None, 16)	80
dense_7 (Dense)	(None, 16)	272
dense_8 (Dense)	(None, 1)	17

Total params: 369 (1.44 KB)

Trainable params: 369 (1.44 KB)

Non-trainable params: 0 (0.00 B)

Edit View Run Kernel Tabs Settings Help

name	Modified
anaconda_projects	5h ago
anaconda3	7h ago
AndroidBuilds	15d ago
AndroidStudioProj...	16d ago
Contacts	9mo ago
CrossDevice	24d ago
DaVinci Resolve M...	3d ago
Desktop	5mo ago
Documents	16d ago
Downloads	7h ago
Favorites	9mo ago
Links	9mo ago
OneDrive	5d ago
Saved Games	9mo ago
Searches	9mo ago
autosave.xmi	2mo ago
import pandas as ...	5mo ago
Untitled.ipynb	9s ago
Untitled1.ipynb	32s ago
Untitled2.ipynb	now

```
Untitled2.ipynb
done = False

while not done:

    action = random.randint(0,1)

    next_state, reward, terminated, truncated, info = env.step(action)

    total_reward += reward

    state = next_state

    done = terminated or truncated

    print("Episode:", episode+1,
          "Reward:", total_reward)

    if total_reward >= 100:
        success += 1

print("\nSuccess Rate:", (success/episodes)*100,"%")

env.close()

WARNING:tensorflow:TensorFlow GPU support is not available on native Windows for TensorFlow >= 2.11. Even
if CUDA/cuDNN are installed, GPU will not be used. Please use WSL2 or the TensorFlow-DirectML plugin.
Episode: 1 Reward: 43.0
Episode: 2 Reward: 16.0
Episode: 3 Reward: 16.0
Episode: 4 Reward: 13.0
Episode: 5 Reward: 15.0
Episode: 6 Reward: 14.0
Episode: 7 Reward: 17.0
Episode: 8 Reward: 17.0
Episode: 9 Reward: 10.0
Episode: 10 Reward: 31.0

Success Rate: 0.0 %
```

Anaconda Assistant

AI-powered coding, insights and debugging in your notebooks.

To enable the following extension create an account or sign in.

✦ Anaconda Assistant 4.18.0

Create Account

Already have an account? [Sign In](#)

❓ For more information, read our [Anaconda Assistant documentation](#).

Would you like to get notified about official Jupyter news?

[Open privacy policy](#) Yes